Response to Reviewers — YOLOv8-FLEO (MDPI Algorithms)

We thank the editor and all three reviewers for their careful and constructive reading of the manuscript. Below we address every comment point by point. Text in [brackets] marks values to be filled from the revised experiments in progress.

Tags: RUN needs a training run WRITE text edit FIGURE figure work

Reviewer 1

RUN **Comment 1.1:** The rationale for setting subspace dimension d=8 is not provided. Please add ablation experiments evaluating algorithm accuracy and hardware cost under different d values to justify the selection of d=8.

Author Response: We thank the reviewer for the comment. We have added an ablation over the per-emotion subspace width d ∈ {4, 8, 16, 32} on RAF-DB, reporting both recognition accuracy and hardware cost (new Table X). Across an 8× range of d, mAP50 remains within a narrow 3-point band (0.790–0.820; mAP50-95 0.759–0.813) with no monotonic trend, indicating that FLEO's accuracy is largely insensitive to the subspace width and that the residual variation is within single-run variance. Hardware cost, by contrast, grows strongly with d: because FLEO projects each of the two neck sites (P3, P4) to K·d = 7d channels, the FLEO footprint is 0.50×, 1.00×, 2.05× and 4.30× at d = 4, 8, 16, 32 (adding 4%, 8%, 17% and 35% over the YOLOv8n backbone). Concretely, each site adds about 11·c1·(7d) + (7d)² parameters, so the FLEO cost scales roughly linearly with d. We therefore fix d = 8 as a balanced, moderate capacity: it preserves accuracy while keeping the K·d projection — and hence the DPU footprint — small, and the larger widths (d = 16, 32) yield no consistent accuracy gain that would justify their 2–4× cost.

d	mAP50	mAP50-95	FLEO cost
4	0.807	0.801	0.50×
8	**0.790**	**0.759**	**1.00×**
16	0.806	0.774	2.05×
32	0.820	0.813	4.30×

Author Action: We have added Table X and a subsection in Section 4 reporting the d-ablation (accuracy and hardware cost), and a sentence in Section 3.1.2 stating that d = 8 is chosen for its accuracy-vs-cost balance given the observed insensitivity of accuracy to d. The training script `scripts/train.py` exposes the width via `--d` for reproducibility.

WRITE **Comment 1.2:** Both RAF-DB and FER2013 are standard single-label benchmarks. How is the fuzzy label vector μ derived from annotator votes as described?

Is it based on multi-annotator metadata from the datasets, or are soft labels generated algorithmically?

Author Response: We thank the reviewer for the comment. The soft labels are generated algorithmically; we do not use multi-annotator metadata. Equation (2) is the general membership definition in which n_c is the vote count for class c. For a single-label corpus this degenerates to $n_c = \mathbb{1}[c = y]$, so Equation (2) reduces to the additively-smoothed soft target $\mu_c = (\mathbb{1}[c=y] + \alpha \cdot \pi_c)/(1 + \alpha)$, computed deterministically from the ground-truth label, the smoothing strength α, and the prior π. The construction is therefore fully reproducible from a single label and does not require annotator votes. The motivation for softening an otherwise one-hot target is that facial expressions are intrinsically ambiguous — confusable pairs such as fear/disgust and happy/surprise share facial action units, and even human annotators disagree — so a hard one-hot target overstates confidence, whereas the smoothed target better reflects this uncertainty and improves calibration and generalization, in line with label smoothing (Szegedy et al., 2016).

Author Action: We have revised Section 3.1.1 to state explicitly that the benchmarks are single-label and that Equation (2) reduces to smoothed soft targets, and we have released the label-construction code in `fleo/fuzzy_loss.py` (activated by the `--fuzzy-alpha` training flag) so the fuzzy targets are reproducible.

WRITE **Comment 1.3:** What is the justification for the values of smoothing parameter α and prior distribution π? Please elaborate the construction pipeline of fuzzy labels and the rationale for parameter selection.

Author Response: We thank the reviewer for the comment. The construction pipeline has three steps: (i) start from the one-hot ground-truth label; (ii) redistribute a fraction α of the probability mass onto the other classes in proportion to the prior π; (iii) renormalise so that μ sums to one. We adopt **$\alpha = 0.1$**, the well-established label-smoothing strength of Szegedy et al. (2016): it is small enough to keep the ground-truth class dominant ($\mu_y \approx 0.92$) yet large enough to inject useful uncertainty; $\alpha = 0$ recovers hard labels and larger α over-smooths the minority classes. We use this standard value rather than tuning α on the benchmarks, to avoid fitting the hyper-parameter to the evaluation data. For the prior we use **π = the empirical class-frequency prior**, so the redistributed mass follows the natural class distribution and is not biased toward rare categories such as disgust.

Author Action: We have revised Section 3.1.1 to describe the three-step construction pipeline and to state and justify the values of α and π.

WRITE **Comment 1.4:** Please ensure all mathematical notations in the equations are clearly defined immediately below the equations.

Author Response: We thank the reviewer for the comment. We agree that every symbol should be defined at its first use.

Author Action: We have added, immediately below each equation, a definition of every symbol (e.g. K, d, μ_c, n_c, α, π, λ_o, λ_b, $\hat{U}$, p, b) in Sections 3.1.1–3.1.3.

WRITE **Comment 1.5:** The English writing needs to improve, and the references need to be properly formatted.

Author Response: We thank the reviewer for the comment.

Author Action: We have carried out a full language edit of the manuscript for grammar and clarity, and we have reformatted all references to the MDPI Algorithms style, ensuring consistency and complete metadata.

Reviewer 2

WRITE **Comment 2.1:** The title should be reduced so that the number of lines of the reduced title is less than or equal to two.

Author Response: We thank the reviewer for the comment.

Author Action: We have shortened the title to "DPU-Native YOLOv8-FLEO: Fold-Out Orthogonalization for Real-Time FPGA Facial-Expression Recognition", which fits within two lines.

WRITE **Comment 2.2:** Delete the paragraph "The remainder of this paper … concludes the study." (lines 100–103), since its contents are redundant.

Author Response: We thank the reviewer for the comment and agree it is redundant.

Author Action: We have deleted the paper-organization paragraph at the end of Section 1.

WRITE **Comment 2.3:** Unify the expression of the same terms (e.g. "YOLOv8-FLEO framework", "YOLOv8-FLEO module", "YOLOv8-FLEO model").

Author Response: We thank the reviewer for the comment.

Author Action: We have unified the terminology to "YOLOv8-FLEO framework" for the overall system and "FLEO block" for the neck module throughout the manuscript.

FIGURE **Comment 2.4:** Check whether the variables X and Y in Figures 1, 2, 3, and 4 are the same; use different symbols if some differ.

Author Response: We thank the reviewer for the comment.

Author Action: We have audited the axis/variable symbols across Figures 1–4 and disambiguated them, using distinct symbols where the quantities differ and a consistent symbol where they are identical.

FIGURE **Comment 2.5:** Redraw Figure 1 (delete "High-Level System Architecture", "YOLO v8" top/bottom, "Proposed", "Block", "Panel B:", "Block Logic (Detailed)"; add the model name in bold at the top of Fig 1(A); remove training flows from 1(B)/1(C); reconcile the Fold-out Graph across panels; remove duplicate items in 1(C)/1(D); use vector graphics; enlarge small text; mention Output in 1(A)).

Author Response: We thank the reviewer for the detailed comment.

Author Action: We have redrawn Figure 1 as a vector graphic: removed the listed labels, added the model name in bold at the top of panel (A), added the Output stage, removed the training flows from the architecture panels, made the fold-out graph consistent across panels, removed the duplicated elements between panels (C) and (D), and enlarged all text to near main-text size.

FIGURE **Comment 2.6:** Remove duplicate words in Figure 2, separate the FLEO Block subfigure, adjust text size/clarity, and merge Figure 2 into Figure 1.

Author Response: We thank the reviewer for the comment.

Author Action: We have merged Figure 2 into Figure 1 as an additional panel, removed the duplicated text, and re-typeset the FLEO block schematic as a clear vector subfigure with enlarged labels.

FIGURE **Comment 2.7:** Redraw Figures 3 and 5 as academic (rather than engineering) figures: add more imagery, remove illustrative text, and do not distinguish box types by colour alone.

Author Response: We thank the reviewer for the comment.

Author Action: We have redrawn Figures 3 and 5 with representative imagery, reduced the explanatory text, and added shape/pattern encodings in addition to colour so the box types remain distinguishable without relying on colour.

WRITE **Comment 2.8:** Remove non-essential text from the "Value" columns of Tables 2 and 3 (e.g. replace "SGD (Momentum = 0.937)" with "SGD").

Author Response: We thank the reviewer for the comment.

Author Action: We have trimmed the "Value" columns of Tables 2 and 3 to concise entries, moving the detailed settings into the surrounding text.

RUN **Comment 2.9:** Open a new section for evaluation metrics; add a heatmap figure; and compare the proposed model with two more models such as YOLO11 and YOLO26.

Author Response: We thank the reviewer for the comment. We agree that a dedicated metrics section and stronger baselines improve the evaluation.

Author Action: We have added a new "Evaluation Metrics" subsection defining every metric, added a confusion-matrix heatmap figure, and added a backbone comparison on RAF-DB: **YOLO11-FLEO reaches mAP50 = 0.802** (vs YOLOv8n-FLEO ≈ 0.79 and the YOLOv8n baseline 0.812). This shows FLEO is backbone-agnostic; we retain YOLOv8n because it is convolution-only and therefore DPU-native, whereas YOLO11/YOLO26 use attention blocks (PSA) that are not compilable by the DPUCZDX8G flow — so, although comparable in accuracy, they cannot be deployed as a single DPU subgraph.

WRITE **Comment 2.10:** Check spelling throughout (e.g. "bottleneckssuch" → "bottlenecks such", "recurrenceand" → "recurrence and", "workarounds").

Author Response: We thank the reviewer for the comment.

Author Action: We have corrected the noted spelling/spacing errors and run a full spell-check over the manuscript.

WRITE **Comment 2.11:** Check whether the affiliations of the first and third authors are correct, since the first half is the same while the second half differs; separate combined affiliations if needed.

Author Response: We thank the reviewer for the comment.

Author Action: We have verified and, where two institutions were combined into one entry, separated them into distinct numbered affiliations for the first and third authors.

Reviewer 3

WRITE **Comment 3.1:** Equation (2) derives fuzzy membership values from annotator vote counts n_c, yet the standard RAF-DB and FER2013 datasets provide a single label. Where do the per-image vote counts come from, and how are the seven-dimensional fuzzy targets generated? This is central to the "Fuzzy Label" contribution and should be fully reproducible.

Author Response: We thank the reviewer for the comment. As also clarified for Reviewer 1, the datasets are single-label and no per-image vote metadata is used. With a single label, $n_c = \mathbb{1}[c = y]$, so Equation (2) reduces to the deterministic smoothed target $\mu_c = (\mathbb{1}[c=y] + \alpha \cdot \pi_c)/(1 + \alpha)$. The seven-dimensional target is thus generated algorithmically from the label, α, and π, and is fully reproducible.

Author Action: We have revised Section 3.1.1 accordingly and released the generating code in `fleo/fuzzy_loss.py` (flag `--fuzzy-alpha`), so any reader can regenerate the fuzzy targets exactly.

WRITE **Comment 3.2:** Section 3.1.3 states that Gram–Schmidt is only a training-time regularizer that can be folded out at inference, yet Table 7 shows that simply removing it collapses RAF-DB accuracy from 0.858 to 0.073. Please clarify the distinction between a training-only regularizer and a forward operator that remains essential until the network is fine-tuned without it.

Author Response: We thank the reviewer for this important observation, which is correct. Gram–Schmidt is not a no-op at inference; naively deleting it does break the forward path (0.858 → 0.073). Our claim is a two-step deployment: (i) fold-out removes the DPU-hostile Gram–Schmidt operator, and (ii) a short post-fold fine-tuning (10 epochs, no orthogonality) lets the remaining DPU-native convolutions re-absorb the function Gram–Schmidt was performing (recovering to 0.849). The benefit is therefore internalized into the weights by fine-tuning, not "already free". We have corrected the wording to avoid describing Gram–Schmidt as a pure training-time regularizer.

Author Action: We have rewritten Section 3.1.3 and the abstract to describe the mechanism as "fold-out followed by post-fold fine-tuning", and we have annotated Table 7 to make the two-step recovery explicit.

WRITE **Comment 3.3:** The authors can refer to some current works, such as: (1) Visually steered reconfigurable intelligent surface-assisted mobile communications; (2) Metasurface Router for Near-Field Multi-User Low-Interference Access Driven by Structured Wave; (3) Azimuth-Dependent Secure Transmission With Orbital Angular Momentum Directional Modulation; (4) Techniques of 2D Human Pose Estimation Based on Computer Vision: A Survey.

Author Response: We thank the reviewer for the suggested references.

Author Action: We have added the four suggested works to the references and cited them in the related-work discussion of reconfigurable/edge systems and computer-vision techniques.

RUN **Comment 3.4:** Ten additional fine-tuning epochs recover accuracy from 0.073 to 0.849. How much would a standard YOLOv8s gain from the same 10 additional epochs and learning-rate schedule? Without this control, part of the recovered performance may come from extra optimization rather than from information internalized through orthogonalization.

Author Response: We thank the reviewer for this valuable point. We have added the requested control: a standard YOLOv8n trained to convergence and then continued for the same 10 additional epochs under the identical schedule ($\eta 0 = 5\times10^{-4}$ cosine, no orthogonality). The control improves by only **+0.2 mAP50 (0.812 → 0.814)**, i.e. a converged detector gains essentially nothing from the extra 10 epochs. The large recovery of the folded FLEO graph therefore reflects genuine re-adaptation of the folded network to the DPU-native operators, not a generic benefit of additional optimization. We further note, in the interest of full transparency, that FLEO does not exceed the plain YOLOv8n detector in raw accuracy (baseline 0.812 vs FLEO ≈ 0.79 mAP50); accordingly we position FLEO's contribution as a **deployment methodology** (train-time orthogonalization that is folded to a DPU-native graph, with the accuracy trade-off Δ_fold quantified), not as an accuracy improvement.

Author Action: We have added the control to Table 7 and revised Section 4 to attribute the recovery correctly, and we have adjusted the abstract and contributions so the claimed advantage is the DPU-native deployment methodology rather than a raw-accuracy gain.

Sincerely,

The Authors